UNIVERSIDAD DE LOS ANDES

Diseño y Programación Orientada a Objetos

Proyecto 1

Análisis, diseño e implementación

Club Deportivo Séneca

Integrantes:

Student-1: Yury Dayana Velasquez Alvarez
Código: 202216733
Correo: y.velasquez@uniandes.edu.co

Student-2: Nelson Fabian Reyes Marquez
Código: 202515993
Correo: n.reyesm@uniandes.edu.co

Student-3: Juan José Sarmiento Bello
Código: 202521072
Correo: j.sarmientob2@uniandes.edu.co

2026

Índice

1. Modelo de dominio **3**

1.1. Descripción 3

1.2. Diagrama de clases del dominio 3

1.3. Entidades principales 5

1.4. Decisiones de modelado 5

2. Requerimientos funcionales **6**

2.1. Usuario final: Socio 6

2.2. Usuario operativo: Empleado 7

2.3. Administrador 8

3. Reglas de dominio **8**

3.1. Deportes de conjunto 8

3.2. Partidos y estadísticas 9

3.3. Competencias 9

3.4. Deportes individuales 10

3.5. Entrenamientos 10

3.6. Tienda y cafetería 10

3.7. Administración 11

3.8. Supuestos de diseño 11

4. Restricciones del proyecto **12**

4.1. Restricciones técnicas 12

4.2. Restricciones de persistencia 12

4.3. Restricciones de acceso y usuarios 12

4.4. Restricciones de alcance 12

5. Diseño e implementación de Entrega 2 **13**

5.1. Arquitectura y responsabilidades 13

5.2. Diagrama de clases de diseño 13

5.3. Secuencias disponibles y consistencia 17

5.4. Decisiones de diseño observables 26

5.5. Implementación, autenticación y persistencia 26
5.6. Ejecución en Eclipse 27

6. Programas de prueba y escenarios pendientes **27**

A. Uso de inteligencia artificial generativa **28**

1. Modelo de dominio

1.1. Descripción

El modelo de dominio representa las entidades del Club Deportivo Séneca, sus atributos, relaciones y jerarquías de herencia. Se organiza en siete subdominios: usuarios y personal, deportes de conjunto, competencias, deportes individuales, entrenamientos, tienda y cafetería, y administración. Los atributos y las multiplicidades de cada entidad se encuentran en el diagrama.

1.2. Diagrama de clases del dominio

El diagrama de alto nivel disponible está en `Diagramas/Diagrama_alto_nivel.drawio.png`; su fuente editable, sin extensión, es `Diagramas/Diagrama_alto_nivel`. La Figura 1 muestra las relaciones de las entidades del dominio en una página horizontal.

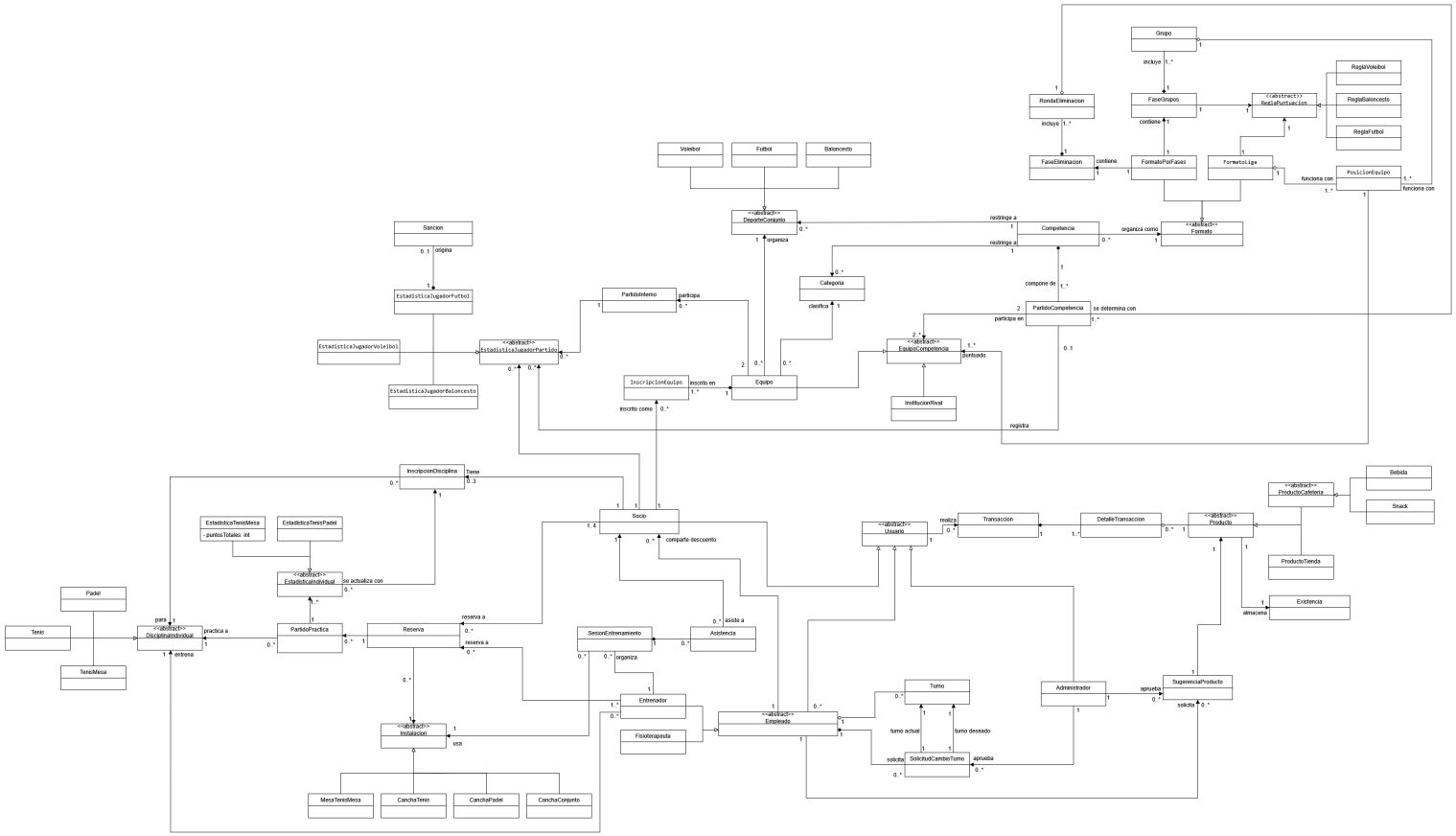


Figura 1: Diagrama de clases de alto nivel disponible para el Club Deportivo Séneca.

1.3. Entidades principales

Tabla 1: Entidades del modelo por subdominio.

Subdominio	**Entidades**
Usuarios y personal	Usuario (abstracta: login, contraseña almacenada, nombre y puntos de fidelidad), Socio (fecha de nacimiento), Empleado (abstracta: descuentos y código), Entrenador (sesiones, disciplinas y reservas), Fisioterapeuta y Administrador.
Deportes de conjunto	DeporteConjunto (abstracta: nombre, mínimo de jugadores) con Futbol, Baloncesto y Voleibol; Categoria (rango de edad); Equipo (nombre, cupo máximo); InscripcionEquipo (posición, número de camiseta); PartidoInterno (fecha, marcador); EstadisticaJugadorPartido (abstracta) con EstadisticaJugadorFutbol, EstadisticaJugadorBaloncesto y EstadisticaJugadorVoleibol; y Sancion (motivo, partidos de suspensión, cumplida).
Competencias	Competencia (nombre, organizador, temporada, nómina mínima), EquipoCompetencia (equipo inscrito), InscripcionCompetencia (equipo y nómina registrada en una competencia), InstitucionRival, PartidoCompetencia (fecha, marcador, escenario), Formato (abstracta) con FormatoLiga y FormatoPorFases (FaseGrupos, Grupo, FaseEliminacion, RondaEliminacion), PosicionEquipo y ReglaPuntuacion (abstracta) con una subclase por deporte.
Deportes individuales	DisciplinaIndividual (abstracta: máximo de instalaciones) con Tenis, Padel y TenisMesa; InscripcionDisciplina (nivel, puntaje de ranking); Instalacion (abstracta: capacidad, techada) con CanchaTenis, CanchaPadel, MesaTenisMesa y CanchaConjunto; Reserva (bloque horario, jugadores, modalidad); PartidoPractica; y EstadisticaIndividual (abstracta) con EstadisticaTenisPadel y EstadisticaTenisMesa.
Entrenamientos	SesionEntrenamiento (fecha, duración, tipo), asociada a una Instalacion, y Asistencia (estado, nota de 1 a 5, comentario).
Tienda y cafetería	Producto (abstracta: nombre, precio) con ProductoTienda (categoría) y ProductoCafeteria (Bebida: es caliente; Snack: alérgenos); Existencia (cantidad y ubicación como texto); Transaccion (fecha, rubro, subtotal, descuento, impuestos, propina, total, puntos) y DetalleTransaccion (cantidad, subtotal).
Administración	Turno (día de la semana, hora de inicio y fin), SolicitudCambioTurno (estado) y SugerenciaProducto (estado).

1.4. Decisiones de modelado

- **Clases de asociación.** InscripcionEquipo e InscripcionDisciplina guardan datos que pertenecen a la relación y no a una sola entidad: un socio puede ser defensa en fútbol y base en baloncesto, o Avanzado en tenis y Principiante en pádel.
- **Sanción como entidad.** Una suspensión afecta partidos futuros y, en competencias, depende de la acumulación durante todo el torneo; por eso no es un atributo de la estadística.

- **Formato separado del organizador.** ASCUN y el Torneo de los Cerros son instancias de Competencia, que funciona con un FormatoLiga o un FormatoPorFases. Su diferencia real es el formato, que el enunciado presenta como habitual y no como fijo.
- **Estadísticas compartidas.** EstadisticaJugadorPartido se asocia tanto a PartidoInterno como a PartidoCompetencia, de modo que las estadísticas por deporte y las sanciones funcionan igual en ambos tipos de partido.
- **Transacciones por rubro.** El atributo *rubro* de Transaccion (tienda, cafetería o mensualidad) permite construir los reportes del administrador sin duplicar clases.
- **Datos derivados.** La edad del socio, la disponibilidad de instalaciones, la tabla de posiciones y el inventario total se calculan a partir de otros datos, por lo que no se almacenan por separado.

2. Requerimientos funcionales

Los requerimientos de esta sección conservan el alcance analizado en Entrega 1; describen el comportamiento esperado y no implican que cada flujo esté disponible desde la consola actual. La implementación efectivamente comprobada y las brechas se distinguen en las Secciones 5 y 6.

Todos los usuarios deben iniciar sesión con login y password (RF-GEN-01); el sistema habilita solo las funcionalidades de su rol. Las reglas citadas (RD) se detallan en la Sección 3.

2.1. Usuario final: Socio

Tabla 2: Requerimientos funcionales del socio.

ID	Nombre	Descripción y validaciones
RF-SOC-01	Consultar mis equipos y partidos	Consulta sus equipos, calendario, resultados y sanciones vigentes.
RF-SOC-02	Inscribirse en disciplina individual	Se inscribe en tenis, pádel o tenis de mesa con nivel inicial Principiante. No puede inscribirse dos veces en la misma disciplina.
RF-SOC-03	Reservar instalación	Reserva un bloque horario indicando jugadores y modalidad. Se valida la capacidad, que no haya traslapes, el cupo de instalaciones simultáneas y la bebida caliente (RD-IND-03, RD-IND-04, RD-TC-06).
RF-SOC-04	Programar partido de práctica	Programa un partido en sencillos o dobles con socios de la misma disciplina. Los niveles deben ser iguales o adyacentes, salvo que participe un entrenador (RD-IND-05).
RF-SOC-05	Registrar resultado de práctica	Registra las estadísticas de cada participante; el sistema actualiza su ranking (RD-IND-06).

ID	Nombre	Descripción y validaciones
RF-SOC-06	Consultar desempeño	Consulta su asistencia, evaluaciones, estadísticas y ranking.
RF-SOC-07	Comprar en la tienda	Compra varios artículos en una transacción, con código compartido y redención de puntos opcionales. Se validan existencias y el cálculo de IVA, descuento y puntos (RD-TC-01 a RD-TC-03, RD-TC-08).
RF-SOC-08	Comprar en la cafetería	Compra bebidas y snacks. Se informan alérgenos, se aplica impuesto al consumo y propina modificable, y se valida la bebida caliente (RD-TC-04 a RD-TC-06).
RF-SOC-09	Consultar compras y puntos	Consulta su historial de compras y su saldo de puntos de fidelidad.

2.2. Usuario operativo: Empleado

Los entrenadores y fisioterapeutas comparten los requerimientos RF-EMP. El enunciado no define funciones adicionales para el fisioterapeuta, cuyo papel principal es la cobertura mínima de turnos (RD-ADM-01).

Tabla 3: Requerimientos funcionales de los empleados.

ID	Nombre	Descripción y validaciones
RF-EMP-01	Consultar turnos	Consulta sus turnos semanales.
RF-EMP-02	Solicitar cambio de turno	Solicita cambiar un turno propio por uno deseado; queda pendiente de aprobación.
RF-EMP-03	Comprar con descuento	Compra en la tienda con su 15 % de descuento, no acumulable (RD-TC-02).
RF-EMP-04	Compartir código	Comparte su código para que otro socio obtenga un 8 % de descuento.
RF-EMP-05	Sugerir producto	Sugiere un nuevo artículo o snack (con alérgenos, si aplica).
RF-ENT-01	Planear sesión de entrenamiento	Crea una sesión con fecha, hora, duración, instalación y convocados (un equipo o un grupo de socios). La instalación debe estar libre (RD-ENT-03).
RF-ENT-02	Registrar asistencia y evaluación	Registra para cada convocado si asistió, faltó o llegó tarde, y opcionalmente una nota de 1 a 5 y un comentario (RD-ENT-02).
RF-ENT-03	Asignar nivel	Asigna el nivel de un socio en una disciplina individual.
RF-ENT-04	Registrar partido interno	Programa y registra un partido entre dos equipos del club, con marcador y estadísticas. Se validan deporte, categoría, mínimo de jugadores y suspensiones (RD-PAR-01 a RD-PAR-05).
RF-ENT-05	Registrar partido de competencia	Registra fecha, rival, escenario, marcador y estadísticas; se actualizan tabla, clasificaciones y sanciones. Solo juegan jugadores de la nómina no suspendidos (RD-COM-03, RD-COM-06).

ID	Nombre	Descripción y validaciones
RF-ENT-06	Consultar analítica	Consulta asistencia, evaluaciones y estadísticas de sus jugadores.

2.3. Administrador

Tabla 4: Requerimientos funcionales del administrador.

ID	Nombre	Descripción y validaciones
RF-ADM-01	Gestionar usuarios	Crea y da de baja socios y empleados. El login debe ser único.
RF-ADM-02	Gestionar equipos	Crea equipos (nombre, deporte, categoría, cupo) y los da de baja conservando su historial.
RF-ADM-03	Inscribir jugador en equipo	Inscribe un socio con posición y camiseta. Se validan edad, cupo, un equipo por deporte y camiseta única (RD-CON-02 a RD-CON-04).
RF-ADM-04	Gestionar competencias	Registra competencias e inscribe equipos con su nómina. Se validan categoría y nómina mínima (RD-COM-02).
RF-ADM-05	Consultar inventario e historial	Consulta instalaciones, existencias por ubicación y el historial de reservas, partidos y sesiones.
RF-ADM-06	Gestionar inventario	Mueve artículos entre ubicaciones y reabastece la tienda. El origen debe tener existencias suficientes.
RF-ADM-07	Resolver sugerencias	Aprueba o rechaza sugerencias de productos; las aprobadas entran al catálogo.
RF-ADM-08	Gestionar turnos	Asigna turnos y resuelve solicitudes de cambio. Solo aprueba si se mantiene la cobertura mínima (RD-ADM-01).
RF-ADM-09	Generar reportes	Genera reportes de ventas por rubro (tienda, cafetería, mensualidades) y periodo (diario, semanal, mensual), y reportes deportivos (asistencia, resultados y rankings).

3. Reglas de dominio

Las siguientes reglas de dominio se derivan de las condiciones y restricciones establecidas en el enunciado del proyecto.

3.1. Deportes de conjunto

- **RD-CON-01:** Cada equipo pertenece a un único deporte (fútbol, baloncesto o voleibol) y a una única categoría (Sub-13, Sub-15, Sub-17 o Mayores).
- **RD-CON-02:** Un equipo no puede superar su cupo máximo de jugadores y la edad de cada jugador debe corresponder a la categoría del equipo.

- **RD-CON-03:** Un jugador pertenece a un único equipo por deporte, pero puede estar inscrito en varios deportes de conjunto.
- **RD-CON-04:** El número de camiseta es único dentro del equipo y la posición debe ser válida para el deporte.

3.2. Partidos y estadísticas

- **RD-PAR-01:** Los partidos internos se realizan entre dos equipos del club que pertenecen al mismo deporte y categoría.
- **RD-PAR-02:** Para disputar un partido interno se debe contar con el número mínimo de jugadores habilitados establecido para cada deporte: 7 en fútbol, 5 en baloncesto y 6 en voleibol. Si un equipo no alcanza ese mínimo, el partido no puede jugarse.
- **RD-PAR-03:** Las estadísticas registradas durante un partido dependen del deporte practicado (goles y asistencias en fútbol; puntos y rebotes en baloncesto; aces y bloqueos en voleibol) y deben permitir realizar posteriormente análisis y rankings.
- **RD-PAR-04:** En fútbol, un jugador que reciba dos tarjetas amarillas en un partido o una tarjeta roja queda suspendido para el siguiente partido de su equipo.
- **RD-PAR-05:** Los partidos de baloncesto y voleibol no pueden terminar en empate. En voleibol, el resultado se determina mediante sets.

3.3. Competencias

- **RD-COM-01:** Una competencia debe definir como mínimo su nombre, organizador, temporada, deporte, categoría y formato. Los rivales de las competencias pertenecen a instituciones externas al club.
- **RD-COM-02:** Un equipo solo puede inscribirse en una competencia cuando pertenece al deporte y categoría correspondientes y su nómina cumple con el número mínimo de jugadores habilitados.
- **RD-COM-03:** Únicamente los jugadores registrados en la nómina de la competencia pueden participar en sus partidos oficiales.
- **RD-COM-04:** En competencias de fútbol se asignan 3 puntos por victoria, 1 por empate y 0 por derrota en las fases de liga o grupos. En baloncesto y voleibol no se permiten empates.
- **RD-COM-05:** El avance de fase se determina de acuerdo con el formato de la competencia y los resultados obtenidos, utilizando los criterios de puntos y enfrentamiento directo cuando correspondan. En una fase de eliminación, el equipo que pierde queda eliminado.
- **RD-COM-06:** Un jugador suspendido no puede participar en el siguiente partido oficial. Las amonestaciones se acumulan según las reglas establecidas para cada competencia.

- **RD-COM-07:** Las estadísticas de los jugadores se acumulan durante las competencias y pueden utilizarse para generar rankings, como goleador o jugador más valioso.

3.4. Deportes individuales

- **RD-IND-01:** En tenis, pádel y tenis de mesa los socios se inscriben de manera individual, sin equipo, y tienen un nivel por disciplina: principiante, intermedio o avanzado.
- **RD-IND-02:** Cada disciplina deportiva individual utiliza instalaciones destinadas para su práctica.
- **RD-IND-03:** Las instalaciones tienen una capacidad máxima de jugadores según la disciplina y modalidad: tenis permite 2 jugadores en modalidad individual o 4 en dobles, pádel permite 4 jugadores y tenis de mesa permite 2 o 4 jugadores según la modalidad. La reserva debe especificar la modalidad correspondiente.
- **RD-IND-04:** Cada disciplina tiene un número máximo de instalaciones en uso simultáneo; si ese cupo está completo en el bloque solicitado, la reserva debe rechazarse.
- **RD-IND-05:** Los partidos de práctica solo pueden realizarse entre jugadores de niveles iguales o adyacentes. Se permite una diferencia mayor cuando participa un entrenador.
- **RD-IND-06:** Cada partido de práctica debe registrar sus estadísticas y actualizar el ranking tipo ELO de los jugadores participantes.

3.5. Entrenamientos

- **RD-ENT-01:** Una sesión de entrenamiento debe registrar fecha, hora, duración, entrenador, instalación y jugadores convocados.
- **RD-ENT-02:** Al finalizar la sesión se registra la asistencia de cada convocado (asistió, faltó o llegó tarde), que puede incluir una evaluación entre 1 y 5 junto con un comentario.
- **RD-ENT-03:** Una instalación no puede tener sesiones de entrenamiento o reservas que se superpongan en el tiempo.

3.6. Tienda y cafetería

- **RD-TC-01:** No se puede realizar una venta por una cantidad superior al inventario disponible.
- **RD-TC-02:** Los empleados tienen un descuento del 15 % y las personas que reciben su código tienen un descuento del 8 %. Estos descuentos no son acumulables con otras promociones.
- **RD-TC-03:** Las compras realizadas en la tienda incluyen un IVA del 19 %.

- **RD-TC-04:** Las ventas de la cafetería incluyen un impuesto al consumo del 8 % y una propina sugerida del 10 % que puede ser modificada por el cliente.
- **RD-TC-05:** Antes de realizar la compra de un snack se deben informar sus posibles alérgenos.
- **RD-TC-06:** No se despachan bebidas calientes a un socio que se encuentre en una cancha de pádel o en una mesa techada de tenis de mesa, ni se permite utilizar dichas instalaciones mientras se lleva una bebida caliente.
- **RD-TC-07:** Toda venta debe registrar su fecha y discriminar sus rubros para efectos contables.
- **RD-TC-08:** Cada venta genera puntos de fidelización equivalentes al 2 % de su valor. Un punto equivale a un peso y los puntos pueden utilizarse en compras posteriores.

3.7. Administración

- **RD-ADM-01:** Durante el horario de atención debe garantizarse la presencia mínima de 2 entrenadores y 1 fisioterapeuta. Los cambios de turno solo pueden aprobarse cuando se mantiene dicha cobertura.
- **RD-ADM-02:** El administrador no puede inscribirse en partidos ni realizar reservas de instalaciones para uso personal.
- **RD-ADM-03:** Las sugerencias de productos realizadas para la tienda o cafetería solo pueden incorporarse al catálogo después de ser aprobadas por el administrador.

3.8. Supuestos de diseño

Para aquellos aspectos del enunciado que no especifican completamente el comportamiento del sistema, el grupo establece los siguientes supuestos de diseño:

- La edad de un jugador se calcula con respecto a la fecha en que se realiza su inscripción al equipo.
- En los partidos internos, las dos tarjetas amarillas recibidas durante un mismo partido generan la suspensión correspondiente. En las competencias, las amonestaciones se acumulan de acuerdo con las reglas de la competencia.
- Para efectos del control de bebidas calientes, se considera que una persona lleva consigo una bebida caliente cuando esta fue adquirida durante los 30 minutos anteriores.
- Una persona se considera dentro de una instalación cuando tiene una reserva o participa en una sesión vigente en dicha instalación.
- En una venta, el sistema aplica los descuentos, posteriormente permite redimir puntos de fidelización y finalmente calcula los impuestos correspondientes.

- Las eliminaciones de equipos o jugadores se manejan de forma lógica con el propósito de conservar la información histórica necesaria para estadísticas, partidos y competencias.

4. Restricciones del proyecto

4.1. Restricciones técnicas

- La aplicación se desarrolla en Java con el paradigma orientado a objetos.
- No se requiere interfaz de usuario: la lógica se demuestra con programas de consola que piden la mínima información (por ejemplo, el nombre de un archivo) y muestran el estado del sistema de forma clara.
- Deben implementarse todas las funcionalidades necesarias para las historias de usuario, aunque no haya interfaz.

4.2. Restricciones de persistencia

- Toda la información es persistente y se guarda en archivos (planos o binarios) dentro de una carpeta distinta a la del código fuente.
- Se asume que solo la aplicación lee y escribe en esa carpeta.
- La información se distribuye en varios archivos por subdominio (usuarios, equipos, competencias, reservas, sesiones, inventario, ventas y turnos), relacionados mediante identificadores únicos.

4.3. Restricciones de acceso y usuarios

- Todo usuario tiene un login único y un password.
- Cada rol (socio, entrenador, fisioterapeuta, administrador) solo accede a sus funcionalidades.

4.4. Restricciones de alcance

- Solo se soportan las funcionalidades descritas en el enunciado.
- No se incluyen pagos reales ni integración con sistemas externos.
- No se registran partidos entre instituciones rivales en los que no participe el club.

5. Diseño e implementación de Entrega 2

Esta sección describe los artefactos que están actualmente en el repositorio y distingue el análisis de Entrega 1 de la cobertura comprobada de la implementación.

5.1. Arquitectura y responsabilidades

El código es un proyecto Java configurado para Eclipse mediante `aplicacionClubDeportivo/ClubDeportivo/.project` y `.classpath`. No se encontró configuración Maven o Gradle. Las fuentes están bajo `src`; los paquetes principales son:

- `club.consola`: Consola, punto de entrada y menús por rol.
- `club.model`: SistemaClub, registro en memoria de usuarios, equipos, competencias, productos, instalaciones y disciplinas.
- `club.modelo.usuarios`: socios, empleados, entrenador, fisioterapeuta, administrador, turnos y operaciones asociadas a esos roles.
- `club.modelo.competencias`: deportes de conjunto, equipos, inscripciones, partidos, competencias, formatos, estadísticas y sanciones.
- `club.modelo.entrenamiento`: disciplinas individuales, sesiones, asistencia, partidos de práctica y estadísticas.
- `club.modelo.instalaciones`: instalaciones concretas y reservas.
- `club.modelo.tienda`: catálogo, existencias, sugerencias, detalles y transacciones.
- `club.persistencia`: ArchivoPlano, carga y guardado del estado en varios archivos CSV, y ManejadorPersistencia, estado serializado (conservada).
- `club.pruebas`: VerificacionEntrega, comprobaciones ejecutables sin JUnit.

La implementación conserva los paquetes `club.model` y `club.modelo`; esa diferencia singular/plural es real y debe mantenerse al importar o citar clases. Consola valida el login y presenta un menú propio para el socio, el empleado (entrenador o fisioterapeuta) y el administrador, con las operaciones de los requerimientos funcionales. Los métodos de dominio no están protegidos de forma transversal por una capa de autorización: el control visible por rol se realiza en los menús de consola.

5.2. Diagrama de clases de diseño

El diagrama de diseño con atributos y métodos se conserva en la fuente editable `Diagramas/Diagrama_de_diseño_con_métodos.drawio` y su exportación PNG; la Figura 2 muestra su versión actualizada con los métodos implementados. El archivo `Diagramas/Proyecto_DPOO_Entrega2.drawio` y su PNG (Figura 3) son otro diagrama detallado, que no se modificó.

Clases y métodos agregados o modificados en la implementación.

Los siguientes elementos se agregaron o cambiaron al implementar el diseño y se reflejan en la versión actualizada de la Figura 2. Las demás clases y métodos permanecen como estaban.

Clase	Métodos agregados o modificados
ArchivoPlano (nueva, `club.persistencia`)	`guardarEstado(SistemaClub): void`, `cargarEstado(): SistemaClub`, `existeEstado(): boolean`, `getCarpeta(): File`. Guarda el estado en varios archivos CSV (uno por clase concreta más `indice.csv`), relacionados por identificadores únicos; carpeta predeterminada `src/datos`.
Administrador	Nuevos: `reporteVentas(SistemaClub, String rubro, LocalDate desde, LocalDate hasta): double`, `historialReservas`, `historialSesiones`, `historialPartidosInternos`, `historialPartidosCompetencia`, `estadoInventario(SistemaClub): String`, `asignarTurno(Empleado, Turno): void`, `listarEmpleados(SistemaClub)`, `consultarSolicitudesPendientes(List<Empleado>): List<SolicitudCambioTurno>`. Sobrecargas que reciben SistemaClub en `crearUsuario`, `crearEquipo`, `crearCompetencia` y `resolverSugerencia`. `resolverCambioTurno(SolicitudCambioTurno, List<Empleado>, boolean): void` conserva su firma; las solicitudes ahora se persisten.
Transaccion	Nuevos: `getPropinaSugerida(): double`, `consultarAlergenos(): List<String>`. Modificado: `calcularPuntosGenerados()` ahora genera el 2 % del valor neto (1 punto = \$1).
Snack	El constructor reemplaza una lista de alérgenos nula por una lista vacía.
DeporteConjunto, Baloncesto, Voleibol	Nuevo `validarMarcador(int local, int visitante): void`: no hace nada en DeporteConjunto; Baloncesto rechaza empates y Voleibol exige un resultado por sets (3 contra 0–2). Lo invocan `registrarMarcador` de PartidoInterno y de PartidoCompetencia.
InscripcionCompetencia (no figura en el diagrama)	Nuevos: `registrarEnNomina(Socio): void`, `enNomina(Socio): boolean`, `getNomina(): List<Socio>`.
Competencia	Nuevos: `registrarNomina(EquipoCompetencia, List<Socio>): void`, `estaEnNomina(Socio): boolean`, `clasificacionGoleadores()` y `clasificacionDestacados()`, ambos `List<Map.Entry<Socio, Integer»`.
PartidoCompetencia	Nuevos: `resolverEmpate(EquipoCompetencia): void` y el privado `registrarSancionDisciplinaria(EstadisticaJugadorFutbol, Socio)`. Modificados: `registrarMarcador(int, int)` (exige nómina y ausencia de sanción vigente) y `getGanador()` (considera el desempate en eliminación).
Grupo	Nuevo privado `enfrentamientoDirecto(EquipoCompetencia, EquipoCompetencia): int`, usado como criterio de desempate al ordenar la tabla.
Asistencia, Socio	Asistencia admite los estados “asistió”, “faltó” y “llegó tarde”; `getPorcentajeAsistencia()` cuenta “llegó tarde” como asistencia; `reservarInstalacion` exige inscripción activa en la disciplina de la instalación.
Consola	Menús por rol (socio, empleado y entrenador, administrador) que cubren los requerimientos funcionales descritos en la Sección 2; carga y guarda el estado con ArchivoPlano y crea datos de demostración.

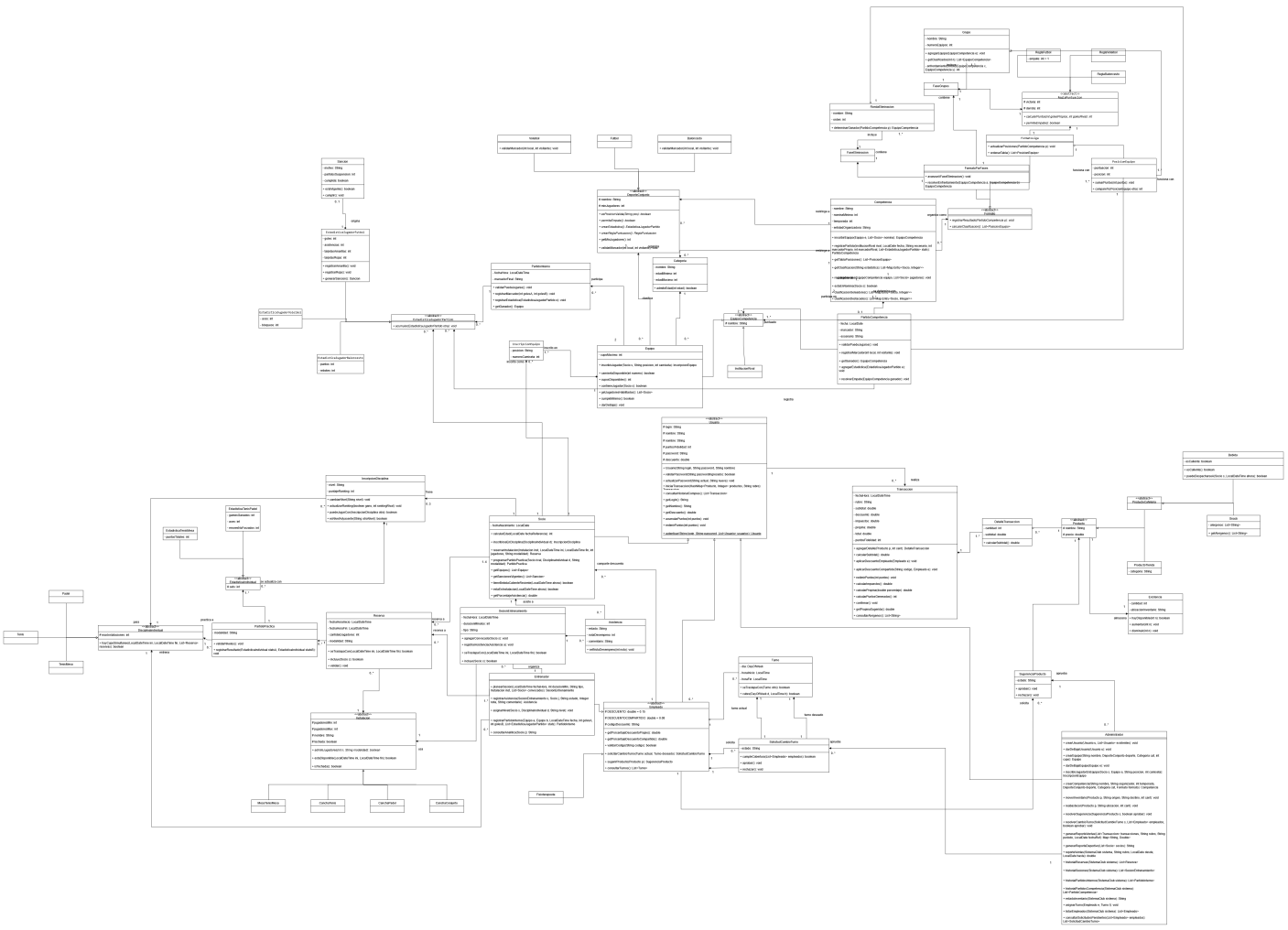


Figura 2: Diagrama de clases de diseño con atributos y métodos.

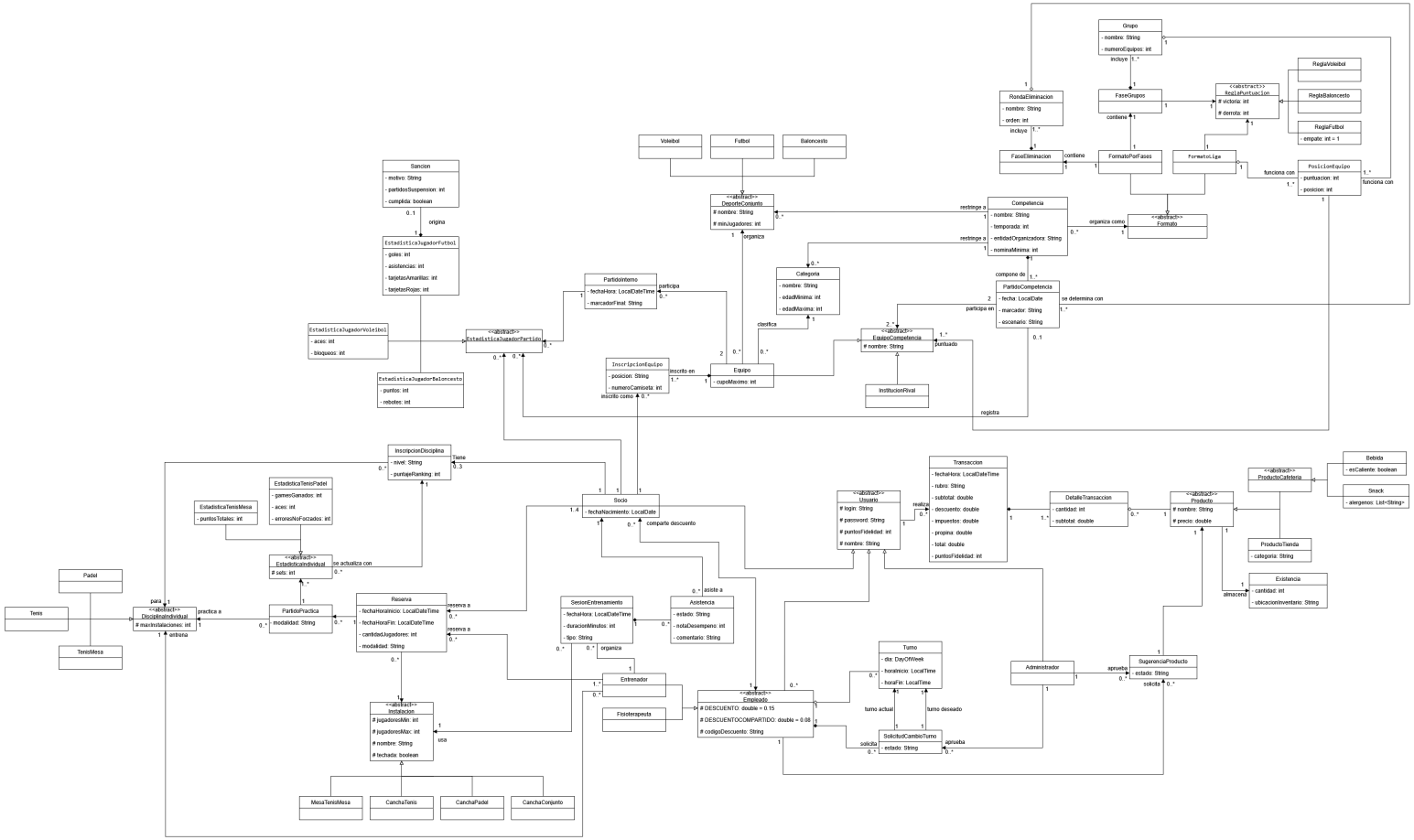


Figura 3: Diagrama detallado adicional de Entrega 2.

Los diagramas de clases disponibles contienen 63 contenedores de clase cada uno. Al compararlos con los 69 nombres de clase de Java, faltan Consola, SistemaClub, ManejadorPersistencia, ArchivoPlano, VerificacionEntrega e InscripcionCompetencia. Esta última es una clase del dominio que debería añadirse al diagrama en una revisión futura. Además, UbicacionInventario, mencionado en la tabla de análisis de Entrega 1, no es una clase Java: Existencia mantiene la ubicación como atributo de texto. La legibilidad a escala de página es limitada por el tamaño de los diagramas; se recomienda verificarlos con zoom en el PDF final.

5.3. Secuencias disponibles y consistencia

La fuente editable `Diagramas/Diagramas de secuencia/Diagramas_de_secuencia(1).drawio` contiene siete páginas; las exportaciones PNG individuales están en su subcarpeta `PNG`. Las Figuras 4–10 integran los flujos existentes, sin rehacerlos. Las secuencias expresan escenarios con validaciones y alternativas, pero algunos mensajes no coinciden con las firmas actuales de Java:

- **Inscripción de jugador:** el flujo de Administrador.inscribirJugadorEnEquipo delega en Equipo.inscribirJugador, que valida edad, cupo, deporte, posición y camiseta. El diagrama menciona operaciones separadas como `admiteEdad`, `cuposDisponibles`, `esPosicionValida` y `camisetaDisponible`, que no existen con esos nombres en el código. Véase la Figura 4.
- **Partido interno:** Entrenador.registrarPartidoInterno invoca PartidoInterno.validarPuedeJugarse, valida estadísticas y registra el marcador. La secuencia menciona llamadas como `getJugadoresHabilitados` y `Sancion.cumplir()`, mientras el código recorre Equipo.getInscripcionesActivas, consulta Socio.tieneSancionVigente y consume sanciones con `Sancion.consumirPartido`; la creación de sanciones está en la validación privada de estadísticas del partido. No debe tomarse como traza literal sin actualizarla. Véase la Figura 5.
- **Partido de competencia:** hay clases y métodos de competencia en el modelo, pero Entrenador no tiene el método de registro de partido oficial que el grupo asigne como actor en una secuencia. Esta invoca mensajes como `registrarPartido(rival, fecha, escenario, marcador, stats)`, `actualizarPosiciones` y `getClasificacion`; el código expone Competencia.registrarPartido(PartidoCompetencia) y no esos nombres. También difiere el actor mostrado. Véase la Figura 6.
- **Reserva:** Socio.reservarInstalacion construye, valida y registra la reserva en el socio y la instalación. El modelo comprueba capacidad/modalidad y traslapes. Véase la Figura 7.
- **Práctica y ranking:** Socio.programarPartidoPractica requiere inscripciones activas y valida niveles; luego PartidoPractica.registrarResultado asocia estadísticas y actualiza el ranking de ambas inscripciones. Véase la Figura 8.
- **Compra:** Usuario.iniciarTransaccion crea la transacción; Transaccion.confirmar valida rubro, inventario, descuentos, puntos, impuestos y restricciones de bebidas calientes. La secuencia pasa `calcularPropina(0.10)`, aunque el método recibe un porcentaje entre 0 y 100 (10 para una propina del 10 %); la consola ya permite ejecutar el flujo desde los menús. Véase la Figura 9.

- **Cambio de turno:** Empleado.solicitarCambioTurno y Administrador.resolverCambioTurno implementan solicitudes y su aprobación con cobertura. La secuencia debe validar sus mensajes contra esas firmas. Véase la Figura 10.

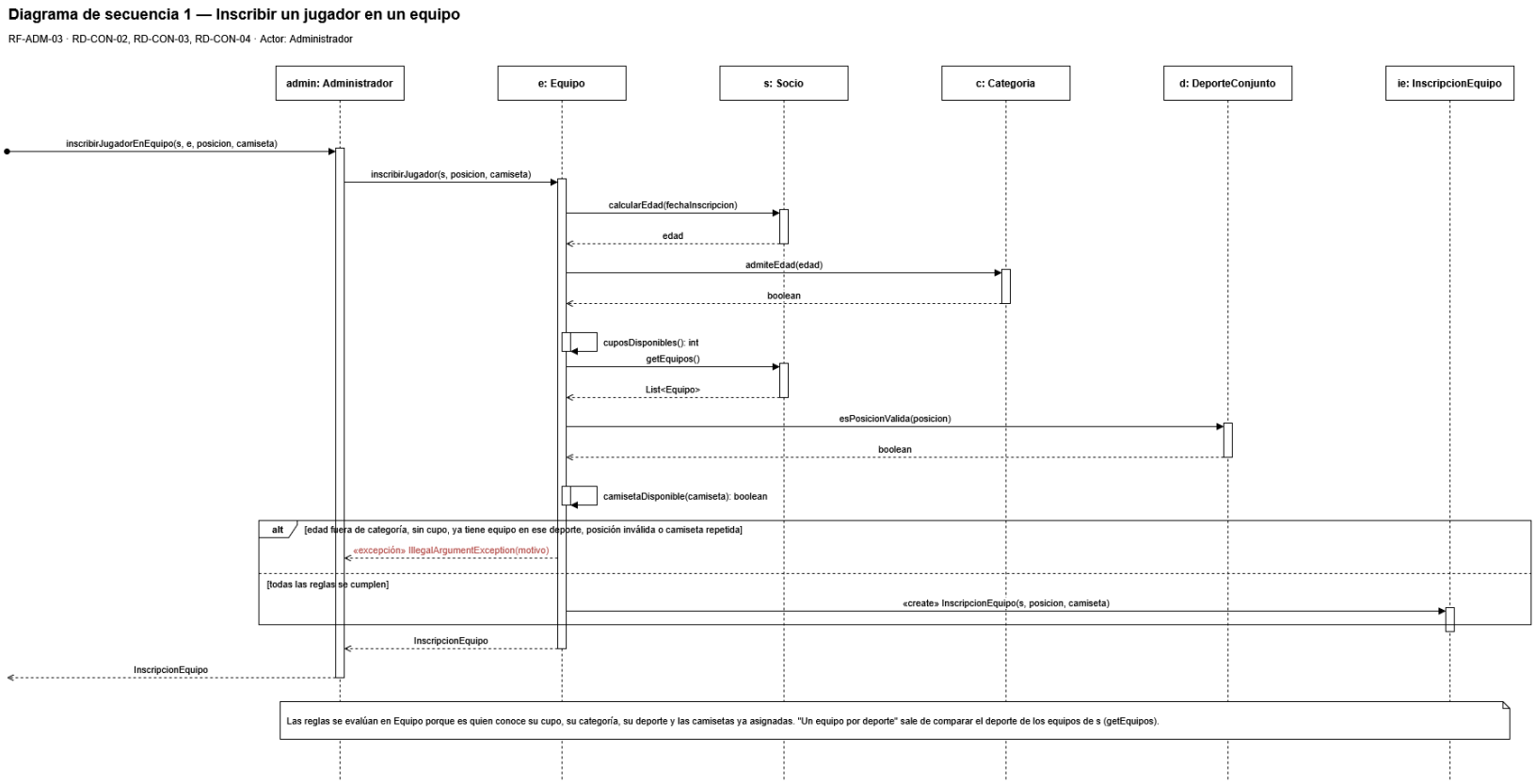


Figura 4: Secuencia disponible: inscripción de jugador en equipo.

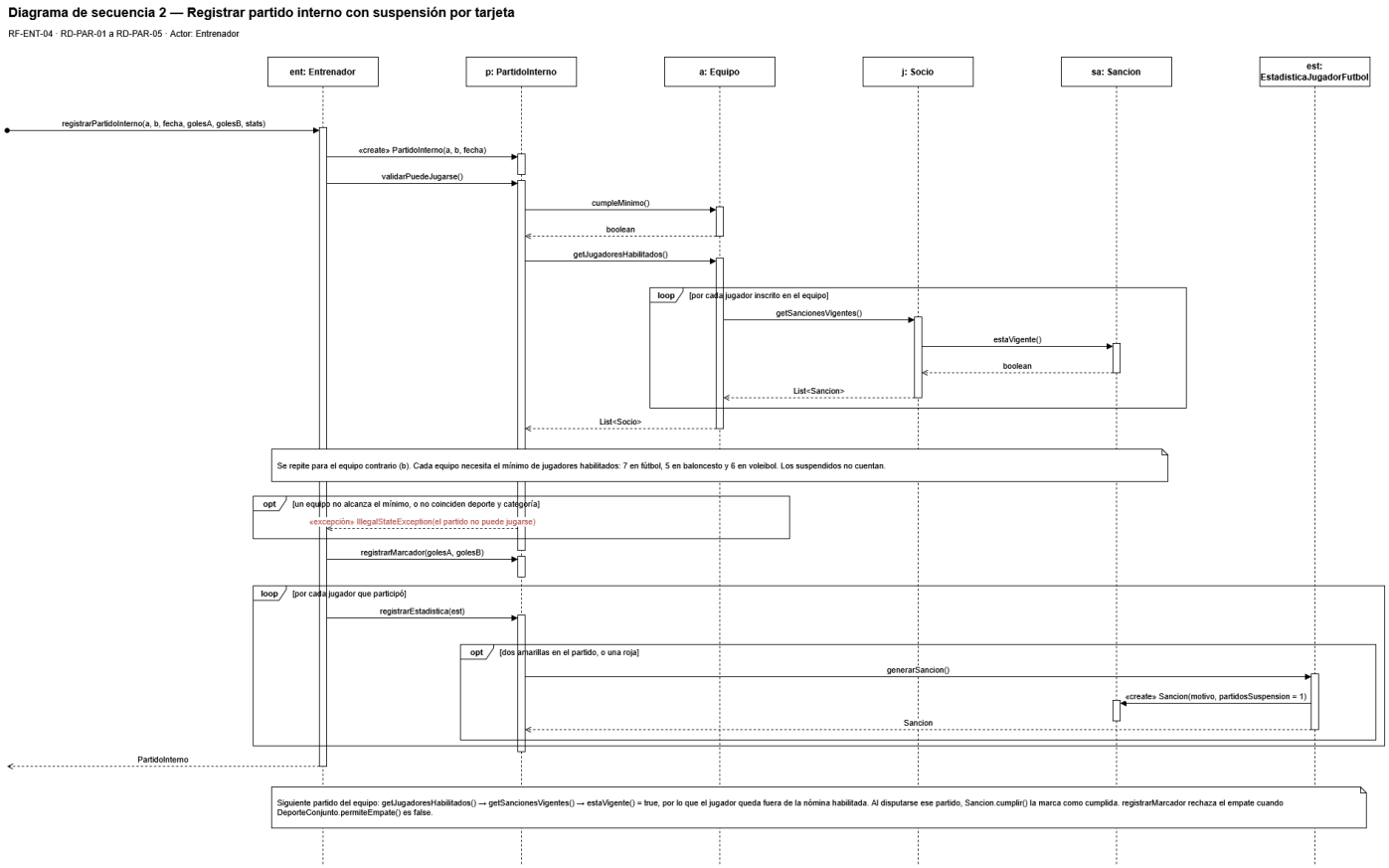


Figura 5: Secuencia disponible: partido interno y sanción.

Diagrama de secuencia 3 — Registrar partido de competencia y actualizar la clasificación

RF-ENT-05 · RD-COM-03 a RD-COM-07 · Actor: Entrenador · Formato: FormatoLiga (Torneo de los Cerros) o FormatoPorFases (ASCUN)

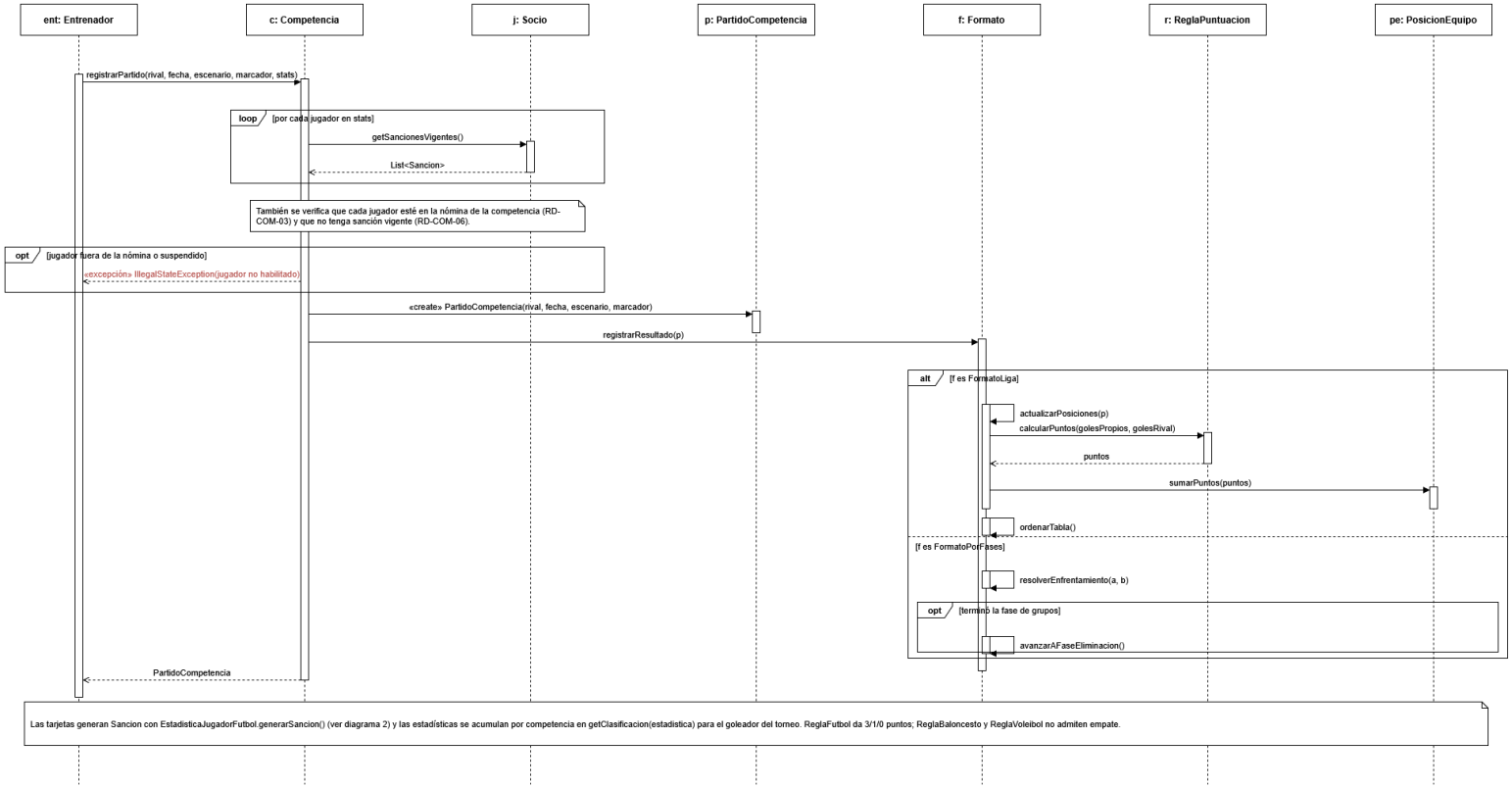


Figura 6: Secuencia disponible: partido de competencia.

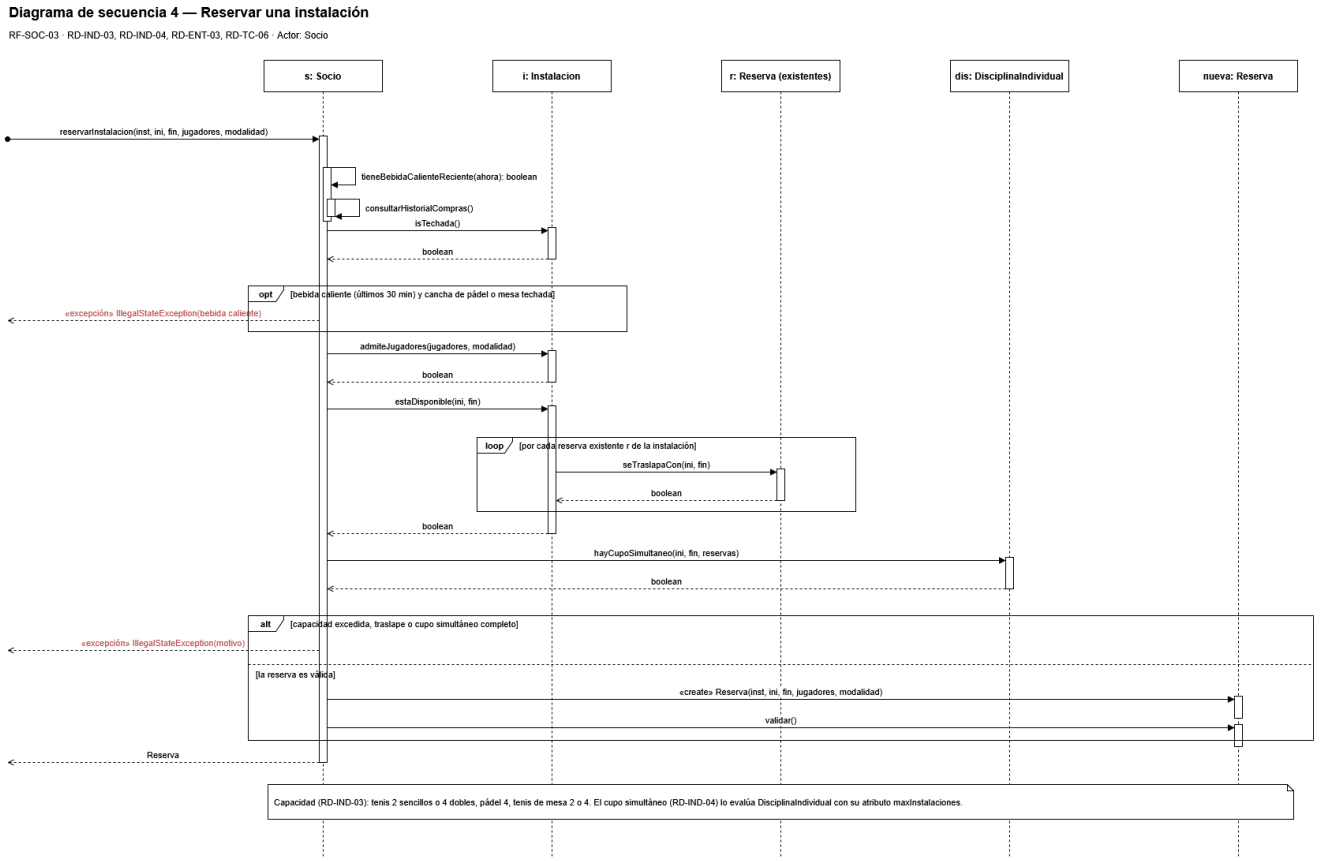


Figura 7: Secuencia disponible: reservar instalación.

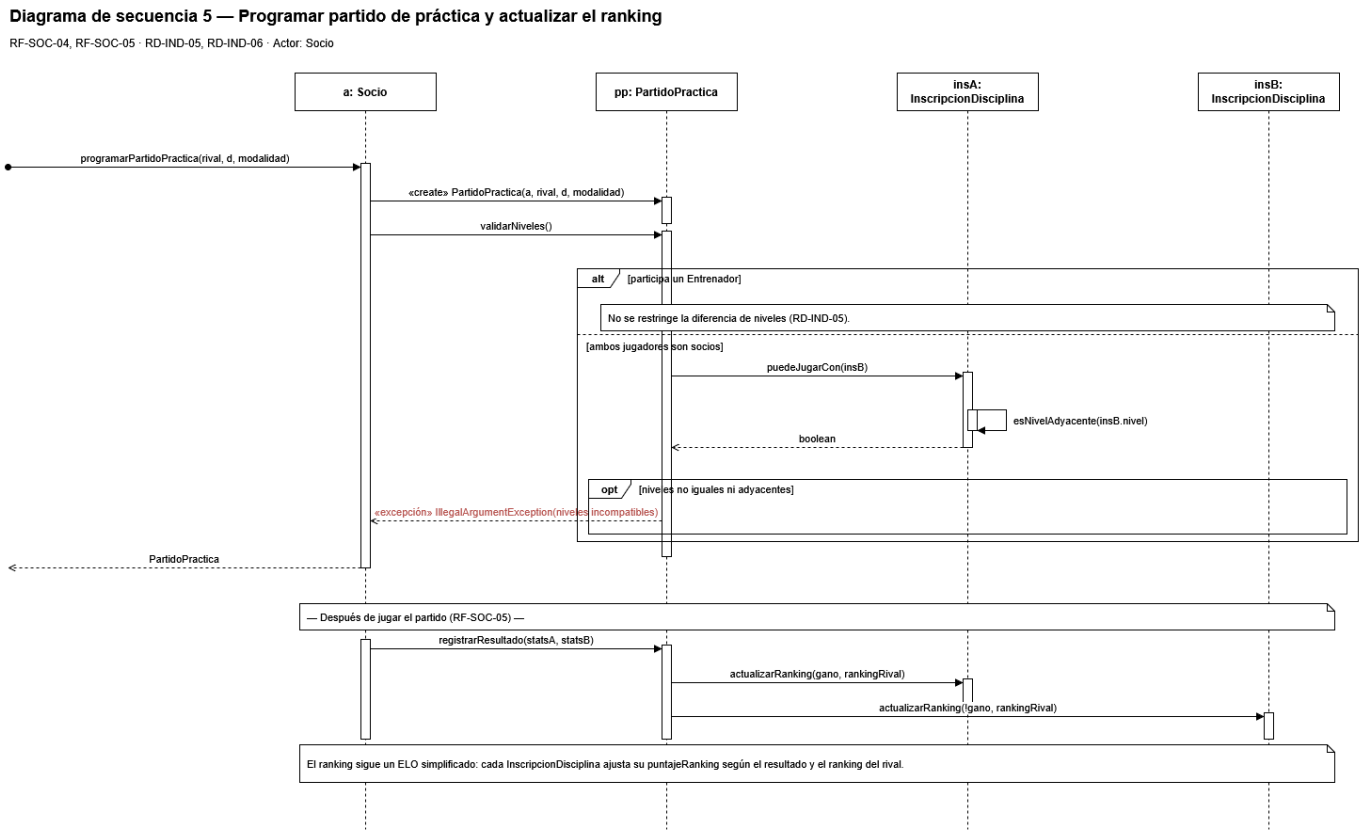


Figura 8: Secuencia disponible: partido de práctica y ranking.

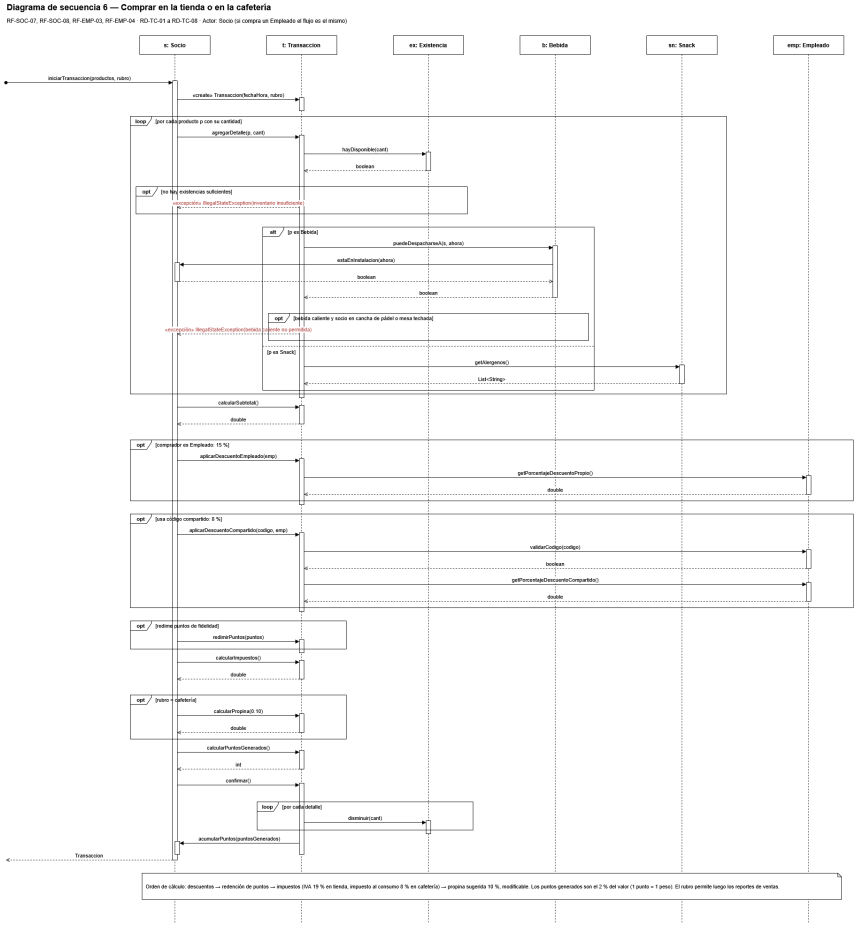


Figura 9: Secuencia disponible: compra en tienda o cafetería.

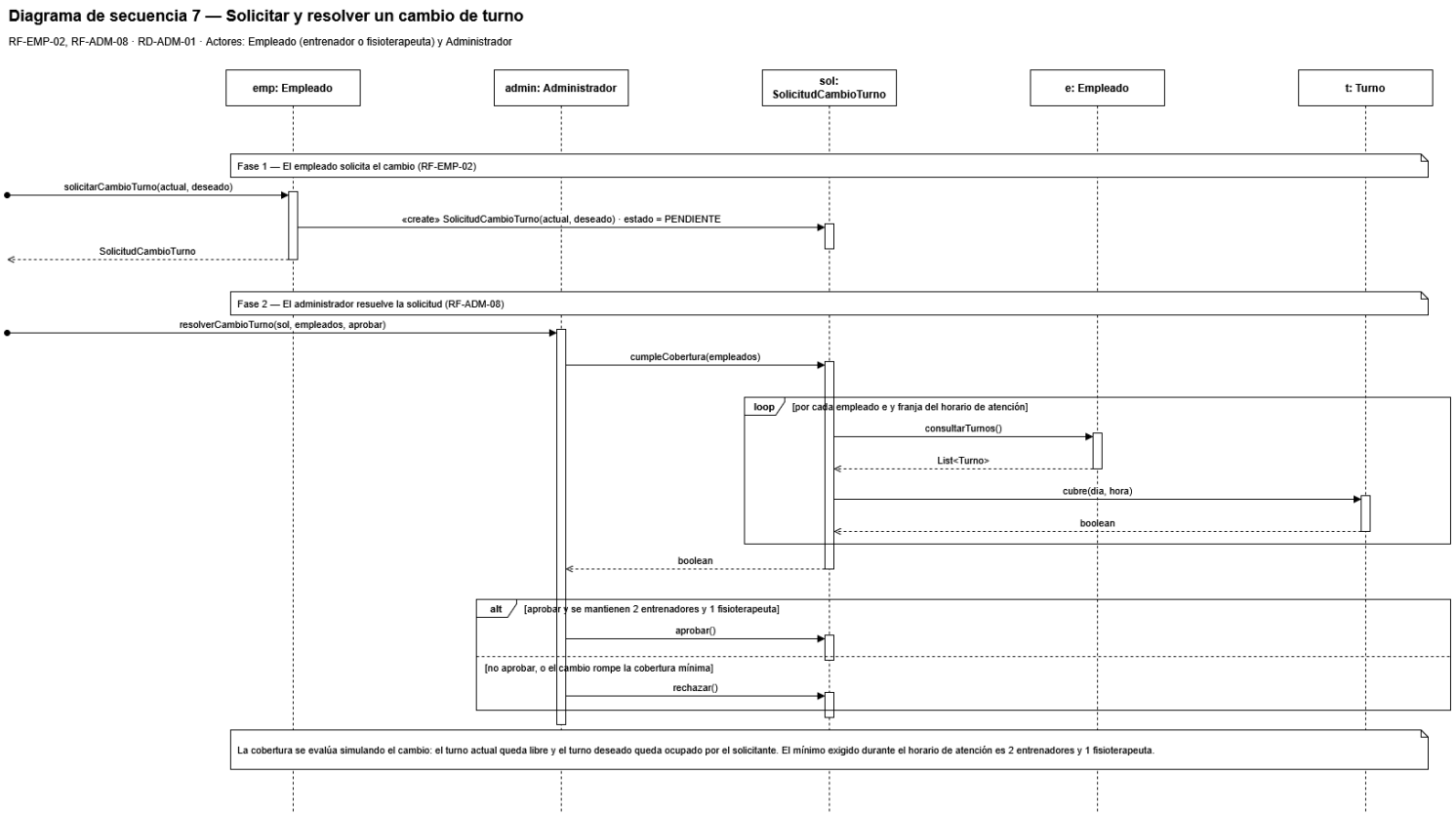


Figura 10: Secuencia disponible: cambio de turno.

5.4. Decisiones de diseño observables

La estructura aplica herencia donde hay comportamientos comunes (Usuario/Socio/Empleado, y clases abstractas para deportes y productos), y usa clases de asociación como InscripcionEquipo e InscripcionDisciplina para guardar atributos propios de la relación. Esta organización reduce la duplicación en el modelo y permite expresar roles y especializaciones; la justificación técnica se infiere de la estructura y debe ser validada por el grupo, pues no se encontró un registro separado de decisiones.
SistemaClub mantiene colecciones del estado en memoria, y ArchivoPlano las escribe en varios archivos CSV, uno por clase concreta más un índice, con identificadores únicos y referencias entre filas por identificador. Así se cumple la restricción de Entrega 1 de varios archivos por subdominio. El guardado escribe primero en archivos temporales y luego reemplaza los definitivos. ManejadorPersistencia, que serializa el grafo completo en un único `club.ser`, se conserva en el código como mecanismo alterno que ya no usa la consola.
La regla RF-TC-08 de Entrega 1 especifica generación de puntos del 2 %. Transaccion.calcularPuntosGenerados calcula ahora el 2 % del valor neto, redondeado, y un punto equivale a un peso, por lo que el código coincide con la especificación.
Las validaciones están ubicadas principalmente en los objetos que conocen la regla: por ejemplo, Equipo.inscribirJugador, Reserva.validar, PartidoInterno.validarPuedeJugarse y Transaccion.confirmar. Las excepciones `IllegalArgumentException` y `IllegalStateException` rechazan entradas inválidas o transiciones no permitidas.

5.5. Implementación, autenticación y persistencia

Consola.main carga el estado con ArchivoPlano (o parte de un SistemaClub vacío si no existe `indice.csv`), completa los datos de demostración y guarda el estado tras cada acción. Esos datos incluyen un administrador, dos entrenadores, un fisioterapeuta, dos socios, turnos, disciplinas individuales con instalaciones, una cancha de conjunto y productos; las credenciales se muestran en consola la primera vez y no se reproducen aquí. El socio accede a los requerimientos RF-SOC, el empleado a RF-EMP y, si es entrenador, también a RF-ENT; el administrador accede a RF-ADM, incluida la resolución de las solicitudes de cambio de turno guardadas.
SistemaClub.autenticar busca login sin distinguir mayúsculas y valida la contraseña. Usuario almacena las contraseñas nuevas como hash PBKDF2 con sal; al validar correctamente un valor serializado antiguo en texto claro, lo migra al formato hash. El menú restringe la creación de usuarios al objeto Administrador, pero no hay una capa general de autorización para invocaciones directas a todos los métodos del modelo. El estado predeterminado crea una cuenta de administrador inicial; no se incluye su contraseña en esta documentación.
Por defecto, ArchivoPlano guarda los datos en `src/datos`. Esa carpeta queda dentro de `src`, mientras el enunciado pide una carpeta distinta a la del código fuente; para cumplirlo basta pasar otra ruta a `ArchivoPlano(String carpeta)` o mover la carpeta. El guardado y la recarga se comprobaron en una carpeta temporal y el estado recargado fue idéntico al guardado.

5.6. Ejecución en Eclipse

1. En Eclipse, importar como proyecto existente la carpeta `aplicacionClubDeportivo/ClubDeportivo`.
2. Ejecutar `club.consola.Consola` como aplicación Java. En la primera ejecución se crea el estado demo en `src/datos` y la consola informa el acceso inicial disponible.
3. Para usar otra carpeta de datos, pasar su ruta al constructor `ArchivoPlano(String carpeta)` en Consola; por defecto se usa `src/datos`. La propiedad `-Dclub.datos` solo aplica a ManejadorPersistencia.
4. Ejecutar `club.pruebas.VerificacionEntrega` como aplicación Java para correr verificaciones sin JUnit. Esta clase crea su carpeta de persistencia temporal y reporta los resultados en consola.

El archivo `readme.md` de la raíz también contiene instrucciones PowerShell para compilar y ejecutar. En esta revisión las fuentes Java se compilaron con `javac` y la verificación informó ocho comprobaciones correctas: autenticación y rechazo de clave incorrecta, hash de contraseña, cupo de equipo, traslape de reserva, venta sin inventario, venta válida con impuesto y persistencia aislada. Estas comprobaciones no cubren todos los requerimientos descritos en la Sección 3.

6. Programas de prueba y escenarios pendientes

Los cuatro programas narrados en la versión previa del documento no están presentes como ejecutables en el repositorio. Se conserva su alcance como escenarios de aceptación pendientes; el ejecutable real es VerificacionEntrega. Los flujos de la consola se recorrieron con entradas de prueba, pero esos recorridos no se entregan como programa. La siguiente tabla separa evidencia de pruebas recomendadas.

Tabla 5: Pruebas disponibles y brechas de demostración.

Programa o escenario	**Estado observado**	**Pendiente**
VerificacionEntrega	Ejecutable; ocho verificaciones de autenticación, hash, cupo, reserva, venta y persistencia aislada.	No es una suite JUnit; faltan asistencia, competencias, sanciones, autorización por rol y turnos.
Persistencia y autenticación	Se prueba autenticación y ciclo guardar/recargar con ManejadorPersistencia en carpeta temporal; el ciclo de ArchivoPlano se comprobó con un programa aparte, no entregado.	Incorporar a VerificacionEntrega la prueba de ArchivoPlano y los accesos denegados por rol.
Deportes de conjunto y competencias	Se prueba límite de cupo de equipo. Inscripción, nómina, partidos, sanciones y tablas se recorrieron desde la consola.	Automatizar edad, camiseta, mínimo de jugadores, sanciones, nóminas, calendario y tabla de posiciones.

Programa o escenario	Estado observado	Pendiente
Deportes individuales y entrenamientos	Se prueba conflicto de horario de una reserva. Inscripción, reserva, práctica, sesión y asistencia se recorrieron desde la consola.	Automatizar modalidad/capacidad, ranking, sesión, asistencia y evaluación.
Tienda, cafetería y administración	Se prueban venta inválida por inventario y venta válida de cafetería. Compras, turnos, reportes e inventario se recorrieron desde la consola.	Automatizar descuentos, puntos, alérgenos, bebida caliente, turnos, cobertura y reportes.

Los casos de la columna Pendiente son recomendaciones de automatización, no pruebas ejecutadas ni programas que ya existan. La consola sí ofrece ahora los flujos funcionales para una demostración interactiva.

A. Uso de inteligencia artificial generativa

Durante la elaboración y actualización de este documento utilizamos una herramienta de inteligencia artificial generativa como apoyo para inspeccionar el código, contrastar las afirmaciones documentales con los archivos disponibles e integrar contenido de la Entrega 2. La herramienta no sustituye nuestra revisión: nos corresponde verificar las discrepancias anotadas, validar el contenido final y confirmar que esta declaración cumple las reglas del curso.
El registro de prompts se encuentra en el archivo `Prompts-IAG.md` en la carpeta Entrega-1 del repositorio.